



FALL 2026

SEG3102 Software Design and Architecture

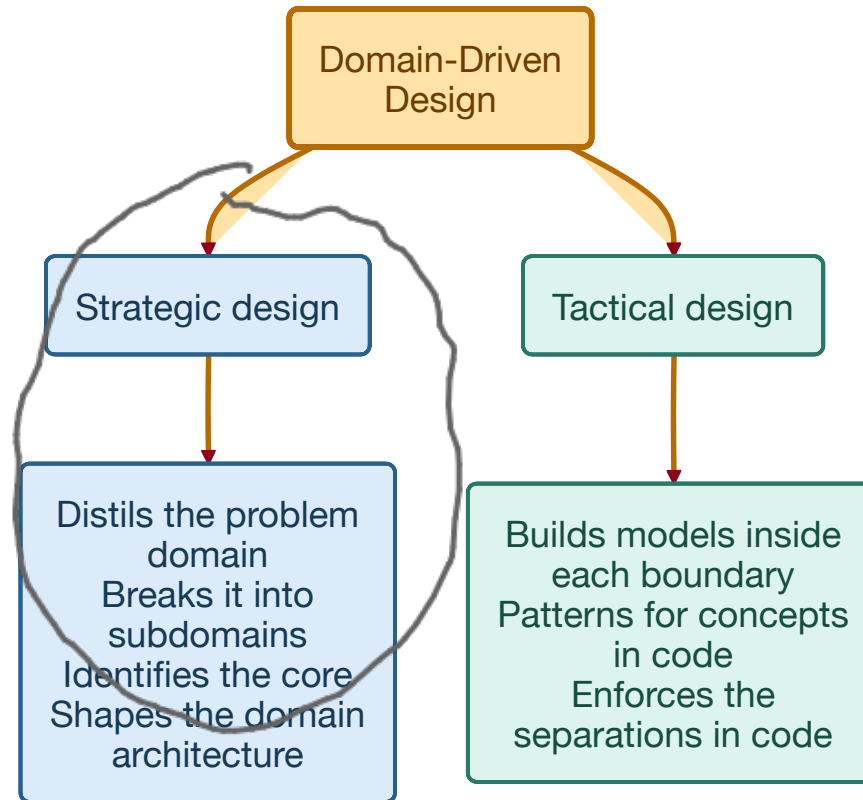
Domain-Driven Design: Strategic Design

Slide content used in this course is adapted from Prof. Stéphane Somé.

Outline

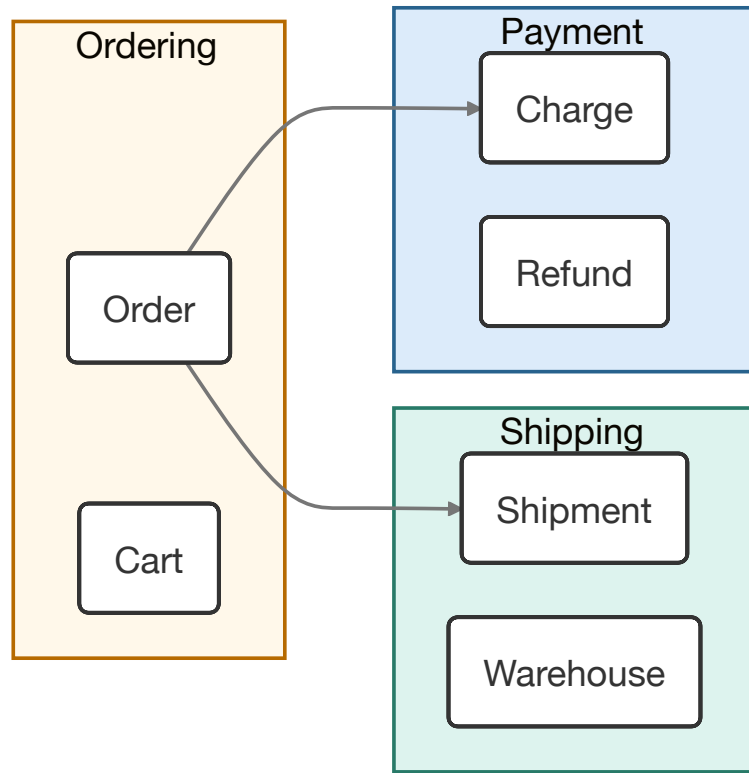
1. Subdomains
2. Subdomains of Online Auction System
3. Ubiquitous language

Two levels of DDD: strategic and tactical



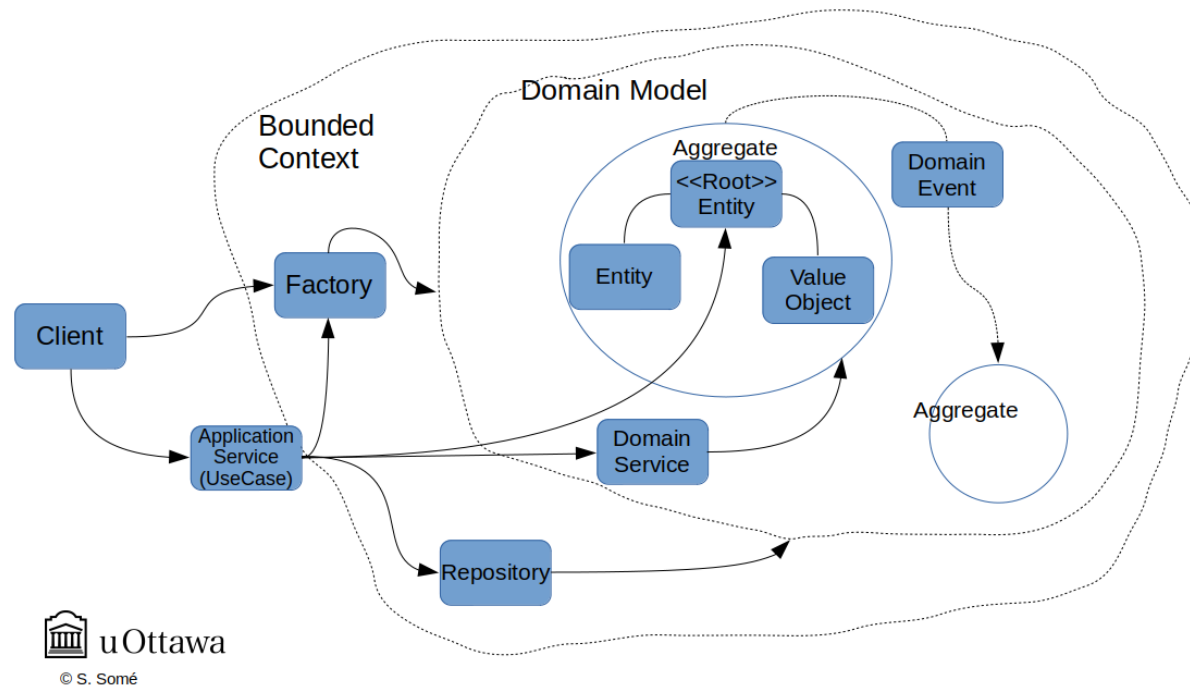
Strategic design **decomposes the domain into boundaries**. Tactical design decides **what the code inside a boundary looks like**.

Strategic design discovers the boundaries



- partition the domain into subdomains;
- rank the partitions by importance to the business;
- draw the boundaries for a single model;
- decide how the boundaries integrate.

Tactical design shapes the code inside a boundary



All the tactical patterns inside one bounded context

For each use case, there is an **application service**:

```

1 // application service
2 class PlaceOrder(/* ... */) {
3     fun execute(id: CustomerId): Order {
4         val cart = carts.of(id)
5         val reservation =
6             inventory.reserve(cart.items)
7         val payment =
8             payments.charge(cart.total)
9         val order = Order.place(
10             cart, reservation, payment)
11         orders.save(order)
12         return order
13     }
14 }
  
```

Subdomains

What is a business domain?

Business domain

A **business domain** is a company's main area of activity: the service it provides to its clients.

-  FedEx provides courier delivery
-  Starbucks is known for its coffee
-  Walmart is a retail chain
- A company can operate in several domains: Amazon does retail and cloud computing
- Domains change: Nokia went from wood processing to rubber to telecoms to phones

Subdomains



We make great coffee. That is the whole business.

Who chose the 30,000 store locations? Who makes coffee? Who does the accounting?



Making coffee is not enough to run a coffeehouse chain.

Real estate, hiring, and finance are each a distinct area of activity related to a coffeehouse business.

Subdomain

A **subdomain** is a fine-grained area of business activity. All of a company's subdomains together form its business domain.

The online shop's areas of activity



Customers fill a cart, pay, and we ship the parcel. That is the business. *→ ordering*

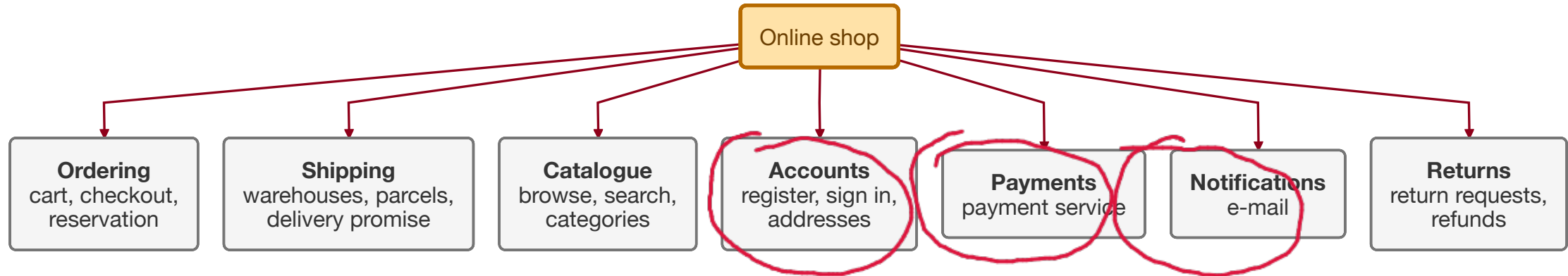
Who keeps the catalogue of products sorted by category? Who registers customers and stores their address and card?



Ah. And we take the payment through a payment service, we e-mail the customer when a parcel ships, and when a customer sends something back we handle the return and the refund. Oh, and we promise a delivery date on every product page.

We can now start to decompose the domain into subdomains

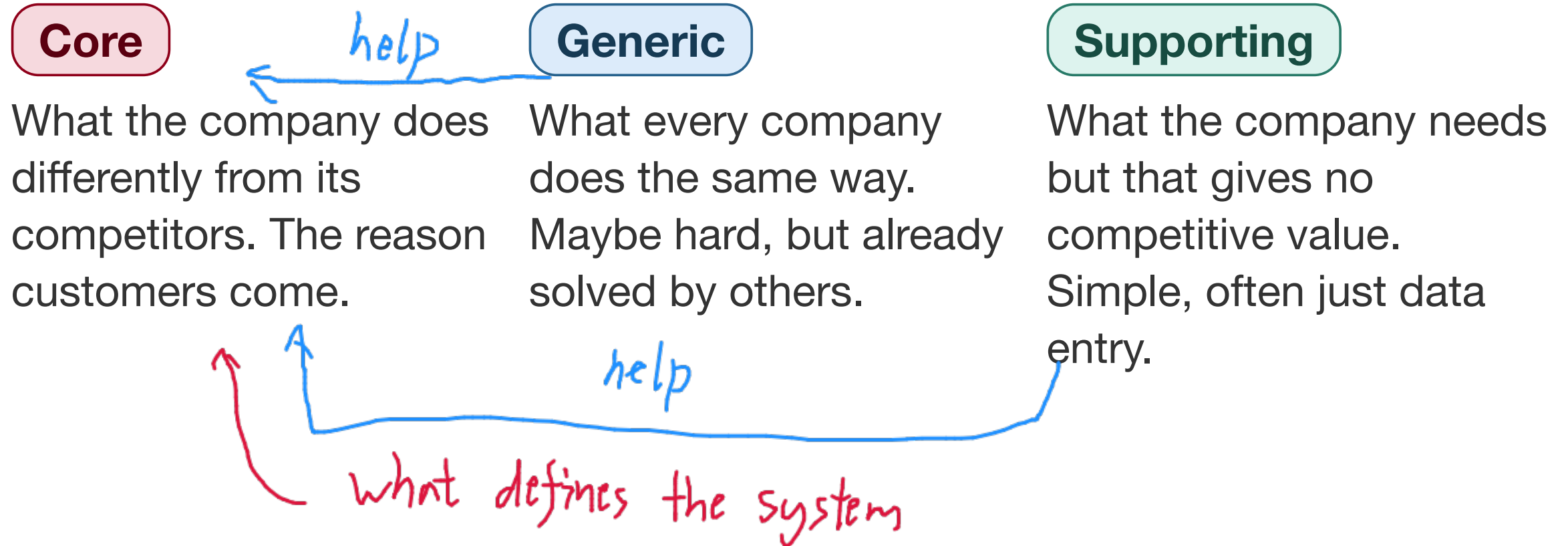
Candidate subdomains of the online shop



- Which of these would you put the most effort on?
- Which would you rather use an existing service?
- Which could be potentially delegated to an intern?

classify

Three types of subdomains



Core subdomains

Core subdomain

A **core subdomain** is what a company does differently from its competitors: a better product, or a cheaper way to deliver one. It is the source of competitive advantage.

- **OpenAI** main core subdomain is its large language model (GPT family).
- **Google Search**'s ranking algorithm earns nothing directly, but it drives the traffic the ads business sells.
- **TikTok**'s algorithm for recommending videos is a core subdomain.

Core is complex, and not always technical

A **core subdomain** typically has **high entry barriers** and is naturally complex.

- Otherwise competitors would just copy it.

They also **change constantly**. If the first attempt is the final one, it was probably not a competitive advantage.

Design
Online shop: What are the most important subdomains to an online shopping business?

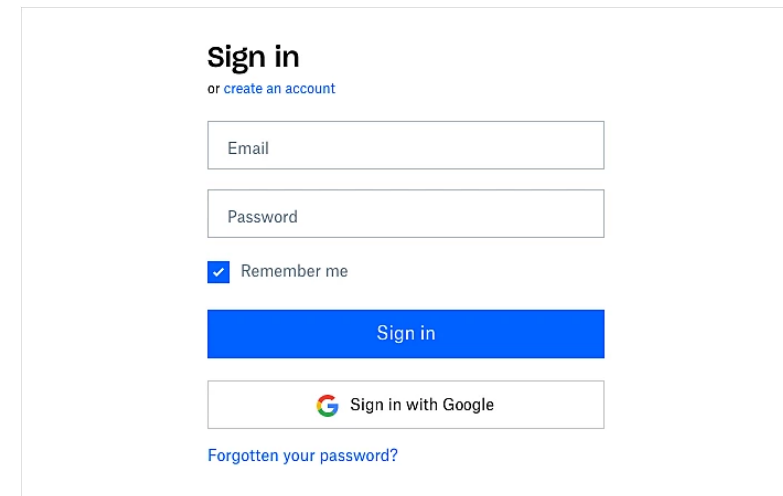
- Shipping
- Returns
- Ordering

Generic subdomains

Generic subdomain

A **generic subdomain** is a business activity that all companies perform the same way. It can be complex, but established solutions exist, and using them gives no competitive edge.

- Authentication and authorisation; encryption, accounting, payment clearing
- Many competitors may even use the same solution



Sign in
or [create an account](#)

Email

Password

☒ Remember me

Sign in

 Sign in with Google

[Forgotten your password?](#)

Online shop: taking **payments** through the payment service, **notification** using e-mail, and (probably) **signing** customers in.

Supporting subdomains

Supporting subdomain

A **supporting subdomain** supports the business but gives no direct competitive advantage. Its business logic is simple.

Ad

Our core is matching ads to visitors. But we also need a catalogue of our banners and landing pages.

These are easy to build, but we just can't find existing solutions that fit our needs.

Online shop: the catalogue of products and maybe some administrative functionalities.

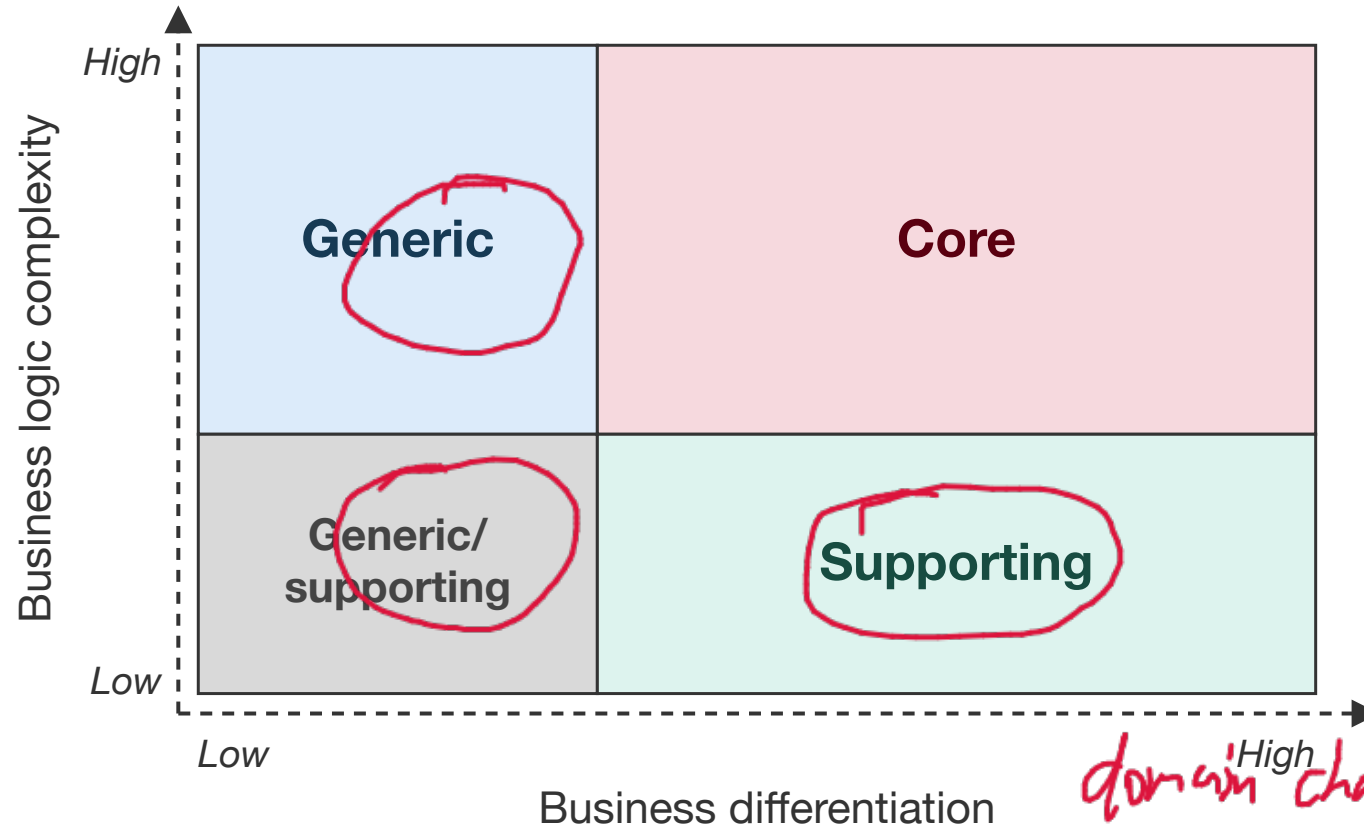
These functionalities are essential, but the owner would not mind a competitor copying them.

Comparing the three types

	Competitive advantage	Complexity	Volatility		Implementation	Problem
Core	Yes	High	High	✓	In-house	Interesting
Generic	No	High	Low	✓	Buy or adopt	Solved
Supporting	No	Low	Low	✓	In-house or outsource	Obvious

- **Complexity:** A core subdomain should ideally be hard to copy; a generic subdomain has existing solution.
- **Volatility:** core subdomains can change constantly depending on the market. Supporting ones rarely change. Generic ones change through patches and new versions.

Differentiation against complexity



- Only **core** is high on differentiation
- **Generic** can be as complex as core, without existing solutions
- **Supporting** is low on both

Gray part: A supporting activity for which a generic product exists. Whichever is simpler and cheaper, ~~integrating or reinventing the wheel,~~ decides the type.

The categorization is not absolute. The type also depends on budget, deadlines, the team you have, and the products actually on the market.

Heuristic for deciding the type

Core or supporting?

Could this be turned into a **side business**? Would someone pay for it on its own?

Yes: core. No: supporting.

Supporting or generic?

Would it be simpler and cheaper to **reinvent the wheel** than to integrate an existing solution?

Yes: supporting. No: generic.

The type decides the solution strategy

	Who builds it	With what	Who works on it
Core	In-house , never outsourced	The most advanced techniques; it must stay easy to change	The most experienced engineers
Generic	Buy or adopt open source	Off-the-shelf	Integration only
Supporting	In-house or outsourced	Work for the core domain	Good training ground for juniors

Exercise: Core, supporting, or generic?

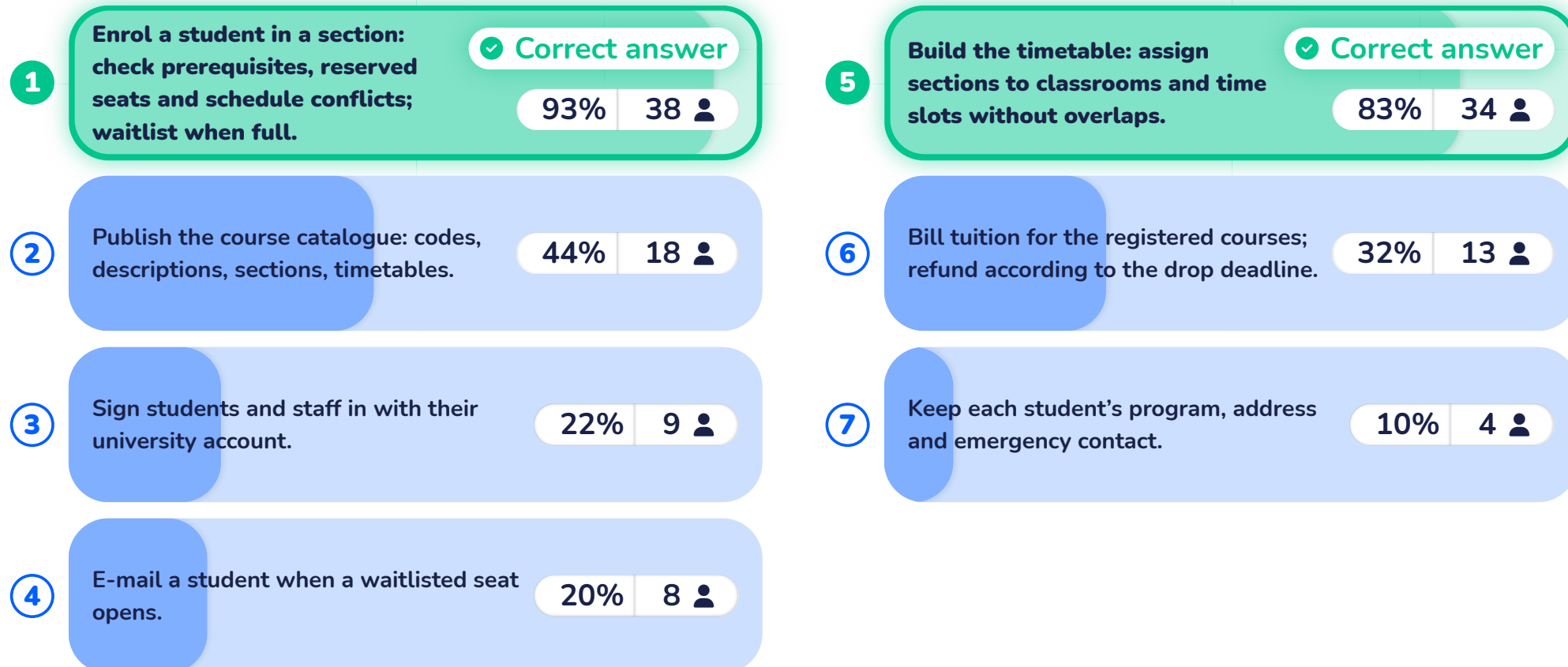
The university builds its own **course registration** system. For each activity of that system, pick **core**, **supporting**, or **generic**.

1. Enrol a student in a section: check prerequisites, reserved seats and schedule conflicts; waitlist when full.
2. Publish the course catalogue: codes, descriptions, sections, timetables.
3. Sign students and staff in with their university account.
4. E-mail a student when a waitlisted seat opens.
5. Build the timetable: assign sections to classrooms and time slots without overlaps.
6. Bill tuition for the registered courses; refund according to the drop deadline.
7. Keep each student's program, address and emergency contact.

Exercise: Core, supporting, or generic?

Go to wooclap.com and use the code **CKCTAGL**

Find all activities that are potentially related to the core subdomain of the course registration system



Exercise: Core, supporting, or generic?

Go to wooclap.com and use the code **CKCTAGL**

Find all activities that are potentially related to the supporting subdomain of the course registration system

1

Enrol a student in a section: check prerequisites, reserved seats and schedule conflicts; waitlist when full.

5%

2



5

Build the timetable: assign sections to classrooms and time slots without overlaps.

3%

1



2

Publish the course catalogue: codes, descriptions, sections, timetables.

✓ **Correct answer**

75%

30



6

Bill tuition for the registered courses; refund according to the drop deadline.

33%

13



3

Sign students and staff in with their university account.

40%

16



7

Keep each student's program, address and emergency contact.

✓ **Correct answer**

50%

20



4

E-mail a student when a waitlisted seat opens.

53%

21



Exercise: Core, supporting, or generic?

Go to wooclap.com and use the code **CKCTAGL**



Find all activities that are potentially related to the generic subdomain of the course registration system

1

Enrol a student in a section: check prerequisites, reserved seats and schedule conflicts; waitlist when full.

0%

0



5

Build the timetable: assign sections to classrooms and time slots without overlaps.

0%

0



2

Publish the course catalogue: codes, descriptions, sections, timetables.

0%

0



6

Bill tuition for the registered courses; refund according to the drop deadline.

0%

0



3

Sign students and staff in with their university account.

0%

0



7

Keep each student's program, address and emergency contact.

0%

0



4

E-mail a student when a waitlisted seat opens.

0%

0



Exercise: Answers

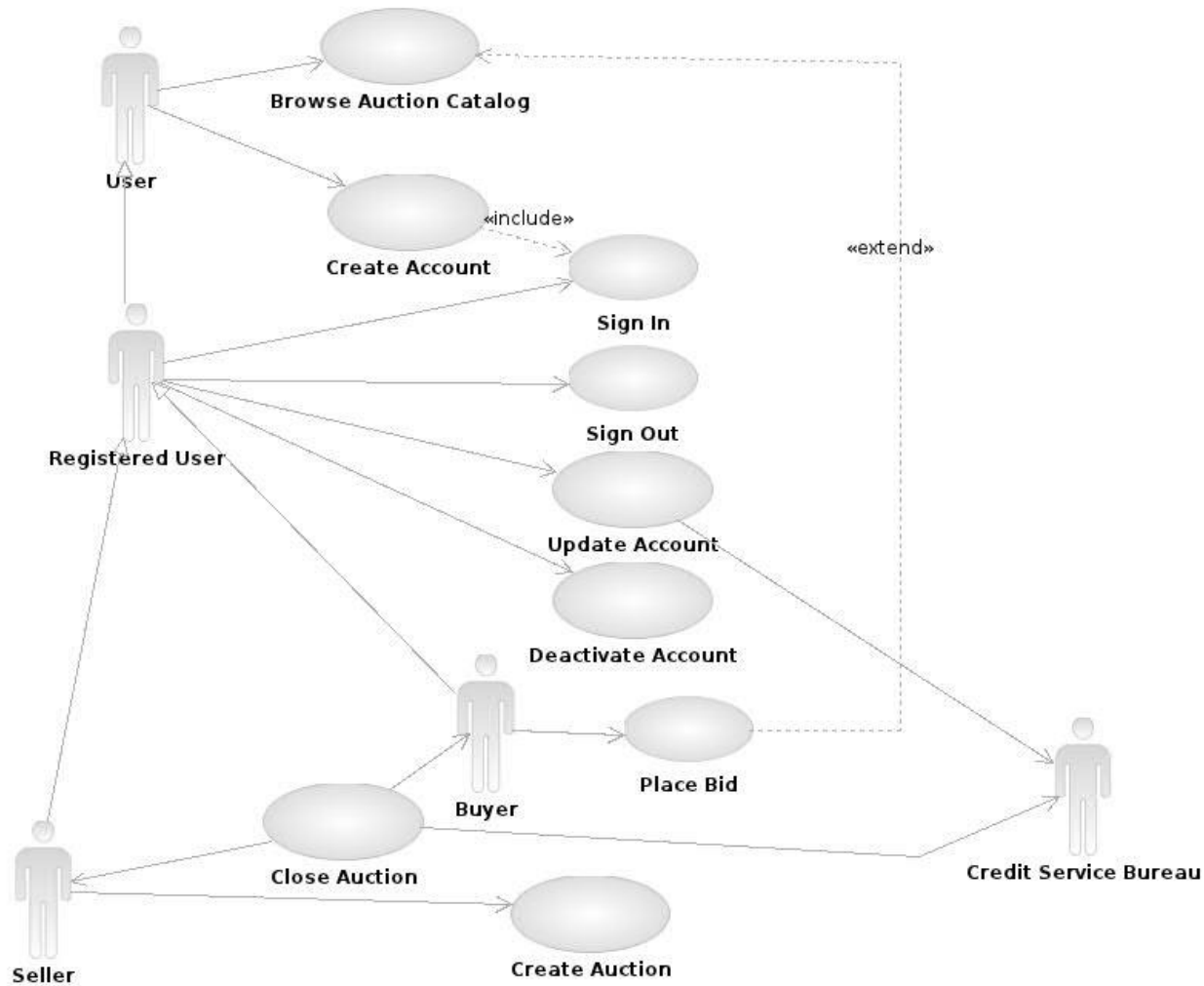
Activity	Type	Why
1. Enrolment	Core	Prerequisites, reserved seats, conflicts, waitlists. Without these components, the system is hard to be called a course registration system
2. Catalogue	Supporting	Browse and search over course data is CRUD
3. Sign in	Generic	Single sign-on; the identity provider already exists
4. Notifications	Generic	Any e-mail service
5. Timetabling	Core or Generic	A hard optimisation problem, so not supporting. Commercial timetabling products exist, but it may not fit the exact needs we have
6. Tuition	Generic	Collecting and refunding money is a solved problem
7. Student record	Supporting	A form and a table

Subdomains of Online Auction System

Online auction system

- Objective: enable buyers and sellers to conduct business transactions in a virtual marketplace
 - Allow sellers to offer items for sale and buyers to bid to purchase the items offered
 - Select highest bidder at the end of an auction period as auction winner

Requirements



Requirements

Use case	Priority	Summary
Sign In	6	The Sign In use case allows the User to identify themselves to the system. If the User already has an account in the system, the User supplies a username and password for authentication purposes. All sign in attempts, whether successful or not, are logged. If the User has pending payment notices, the system informs the User that payment for the notices must be made before the User can participate in any auction (as either the Buyer or the Seller).
Sign Out	9	The Sign Out use case allows the User to indicate that they no longer wants to use the system. The User's account remains unchanged (that is, it is not removed). Sign out events are not logged. The User must be signed in to sign out (see the Sign In use case.)
Browse Auction Catalog	4	The Browse Auction Catalog use case allows the User to browse items currently available for auction. The User may look at all of the items currently available for auction or search based on category. The User has the option to place a bid on an item available for auction (see the Place Bid use case). The User does not have to be signed in to browse the auction catalog.

Requirements

Use case	Priority	Summary
Place Bid	2	The Place Bid use case allows a Buyer to place a bid on an item when browsing the auction catalog (see the Browse Auction Catalog use case). To place a bid on an item, the Buyer must be signed in (see the Sign In use case) and must not have any pending payments (see the Close Auction use case).
Create Auction	1	The Create Auction use case allows a Seller to create an online auction by specifying auction information (including the start and end times) and, if it was not provided when the Seller created their user account, credit card information (see the Create Account use case). Once the auction has been created, Buyers can then place a bid on the item (see the Place Bid use case). To create an auction, the Seller must be signed in (see the Sign In use case) and must not have any pending payments (see the Close Auction use case).
Close Auction	3	The Close Auction use case describes what occurs when an auction is closed. This use case is executed when an auction's end time is reached. When an auction is closed, the current highest bid becomes the purchase price for the item. The Buyer and Seller are notified that the auction has closed, what the final purchase price is, and information on how to contact each other in order to complete the transaction. The transaction fee is charged to the Seller's credit card. If the Seller's credit card cannot be charged, a pending payment is recorded for the Seller and the Seller will not be able to participate in any auctions until the pending payment is made.

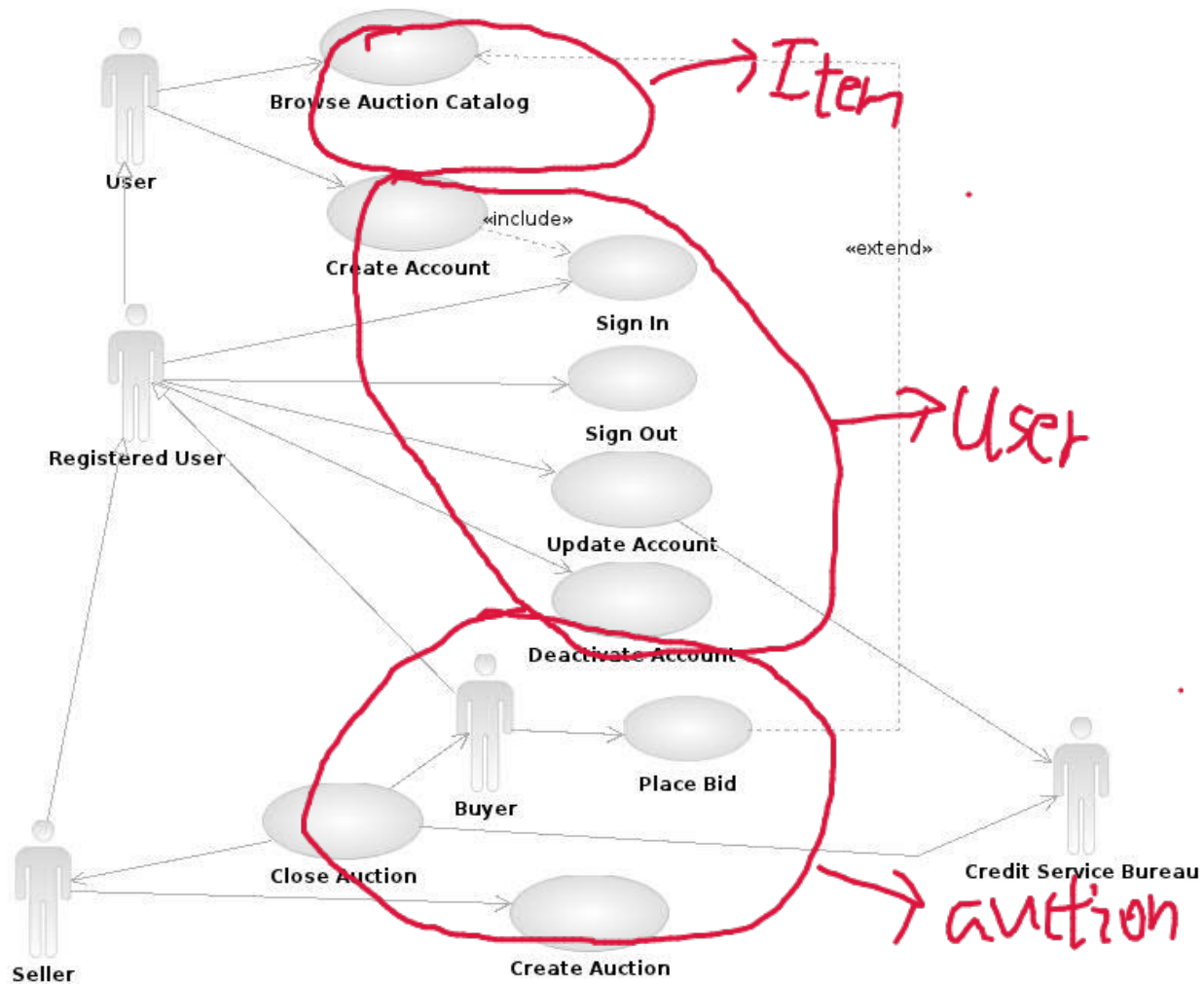
Requirements

Use case	Priority	Summary
Create Account	5	The Create Account use case allows the User to create and activate a user account, using entered user information, which optionally includes credit card information. Once the account has been created and activated, the User is considered to be signed in.
Update Account*	7	The Update Account use case allows a User to update the information maintained in their account. If new credit card information is entered, the system attempts to pay any pending payment notices via the Credit Service Bureau using the entered information.
Deactivate Account	8	The Deactivate Account use case allows a User to deactivate their account. If there are any auctions in progress for the account, or the User has any pending payments, the account cannot be deactivated. A deactivated account is not deleted from the system (it may be needed later for auditing purposes), but the User cannot access it.

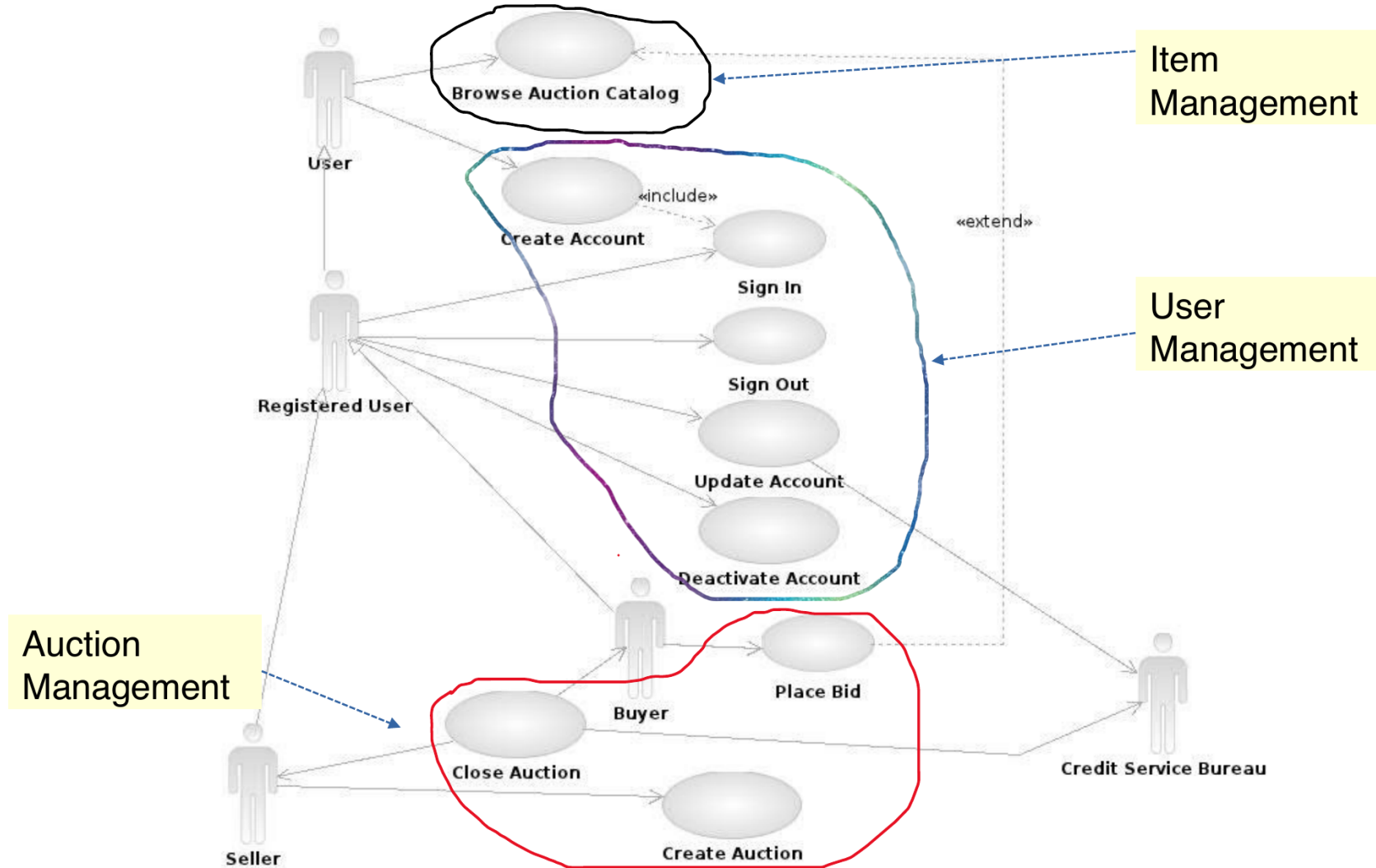
Sub-domains

- Distinct areas of
 - Capability
 - Functionality
 - Concerns
 - ...
- Initial determination
- Derived from understanding of Domain/ Requirements
 - domain description
 - use cases
- Single Responsibility

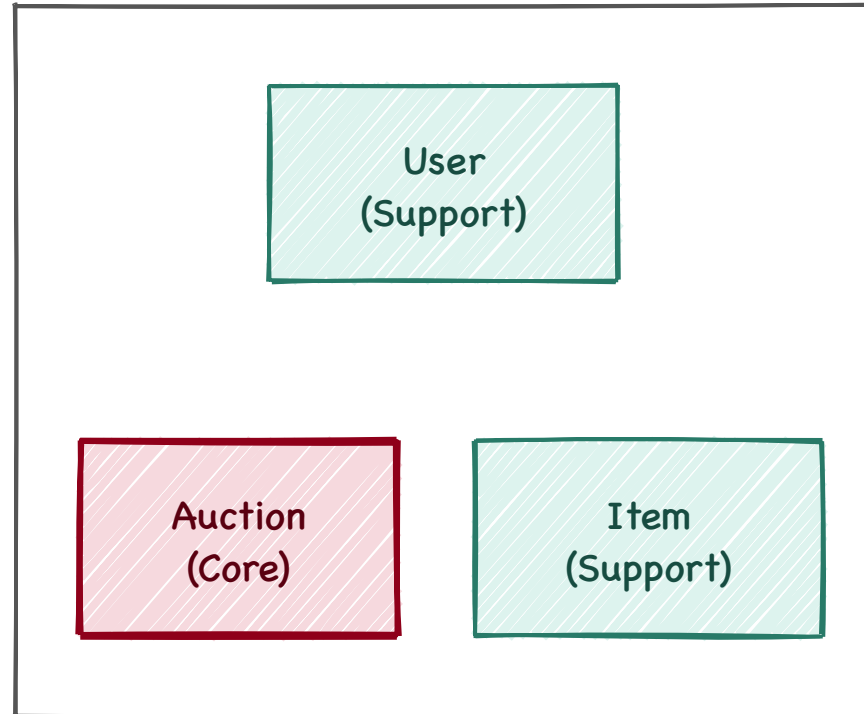
Sub-domains



Sub-domains



Sub-domains



Sub-domains

Sub-domain	Type	Description	Rationale
Auction	Core	Concepts related to auctions. Captures the information associated to auctions and processes including bidding as closing of auctions.	Creating and managing auctions is the main objective of the system. This domain mainly supports use cases: Create Auction, Place Bid, Close Auction, Browse Auction Catalog. These are the highest-prioritized use cases.
User	Support	Concepts related to the users and their roles. Responsible for the registration of users, role allocation to users, signing in, signing off, ...	This domain mainly supports use cases: Create Account, Sign In, Sign Out, Update Account, Deactivate Account. Many systems register and manage users. It is likely that reusable solutions exist for this aspect of the application.
Item	Support	Concepts related to items. Provides support for browsing and search based on specific product category.	This domain mainly supports use cases: Create Auction, Browse Auction Catalog.

Sub-domains maps to code structure

```
1 auction-system
2   └─ src/main/kotlin/seg3x02.auctionsystem
3       └─ domain
4           ├── auction
5               └─ README.md
6           ├── item
7               └─ README.md
8           └─ user
9               └─ README.md
10  └─ AuctionSystemApplication.kt
```

domain/auction/README.md

```
1 ## Auction Sub-Domain
2
3 - Core
4 - Auctions, Bids
```

Tutorial repository

<https://github.com/SEG3102-Fall2026/auction-system-tutorials>

Branch: `sdomains`

Now, how to we ensure the implementation of the sub-domains is consistent with the requirements and domain knowledge?

Ubiquitous language

Translation loses information



When a customer sends an item back, we refund them once the warehouse has checked it.

Got it: a return request requires an authorization, then a memo after inspection.



So I need a `ReturnDTO`, a `RefundProcessorImpl`, and an `inspected` flag on the item row.

These are three sets of vocabularies for one process. Each translation drops a little meaning, and six months later nobody remembers what “return” meant exactly.

Ubiquitous language

Ubiquitous language

The **ubiquitous language** is the common vocabulary of the domain, shared by domain experts and developers, and used in conversations, documents, models, and code.

- **Common terminology** and structure for the whole team
- Binds the **model and the implementation**
- Supports **change**: when the business changes a term, the code follows

The domain model **is** the ubiquitous language. Diagrams, use cases, and classes in code are different views of the same vocabulary.

Binding model and implementation

The use case says

```
1 4. The system reserves the  
2   items in inventory.
```

The code says

```
1 stockManager.decrementQty(sku, n)  
2 flags[sku] = "HOLD"
```

- “reserve” is what the domain says
- **Reservation** is now a common concept
- “HOLD” flag and “decrement” are solution terms

After aligning the language

```
1 val reservation =  
2   inventory.reserve(order.items)
```

If the domain expert cannot read a method name aloud, the name is probably wrong.

Signs the language is missing

- **Two words for one thing:** “client” in the UI, “customer” in the database, “user” in the code
- **One word for two things:** “order” means both a cart at checkout and a shipment
- **Generic names:** Manager, Helper, Processor, Data
- Requirements and code written in heavy technical jargon
- Developers constantly translating meanings to the domain experts

The language is built through **collaboration** between domain experts and developers

Exercise: What are good candidates for the ubiquitous language vocabularies?

Which terms below are good candidate for the **ubiquitous language** of the online shopping system?

- Customer
- Item
- Transaction
- Order confirmation
- Cart
- Status

Exercise: What is in the model?

Go to wooclap.com and use the code **CKCTAGL**

Which terms below are good candidates for the
****ubiquitous language**** of the online shopping system?

- ① Customer 0% 0 
- ② Item 0% 0 
- ③ Transaction 0% 0 
Click on the projected screen to start the question
- ④ Order confirmation 0% 0 
- ⑤ Cart 0% 0 
- ⑥ Status 0% 0 

Exercise: Answers

Potentially good candidates

- Customer
- Order confirmation
- Cart

Potentially bad candidates

- Item (too generic, is it “order item” or “product”?)
- Transaction (potentially mixed with technical meaning: “payment” or “database transaction”?)
- Status (too generic if used along. Is it “order status” or “shipment status”?)
 - Needs to be used within a context to be meaningful

Tool for building the ubiquitous language: a glossary

Term	Meaning
Order	A customer's confirmed purchase: order lines, delivery address, payment, one or more shipments
Order line	One product on an order, with its quantity and the price charged at the time of purchase
Reservation	Stock set aside for an order before the payment is confirmed; released if the payment fails
Delivery promise	The date by which the shop commits to deliver, from the warehouse and the destination
Return window	The 30 days after delivery during which an item can be returned

The glossary is a living document. It is updated as the project progresses, and as the domain experts and developers learn more about the domain.

Tool for building the ubiquitous language: Gherkin

Describing system behavior using the ubiquitous language

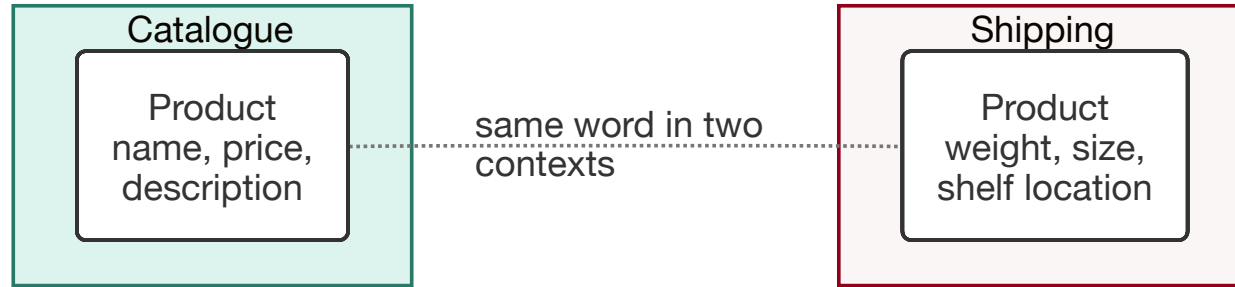
```
1 Scenario: An order is not placed
2     when stock runs out
3     Given 5 "Wireless mouse" available
4         in inventory
5     And a customer with 6 "Wireless mouse"
6         in the cart
7     When the customer places the order
8     Then the order is not placed
9     And 5 "Wireless mouse" remain available
```

The shop owner can read this and say “yes” or “no”. That is the point.

These scenarios can directly be mapped into automated tests to make the **ubiquitous language** executable.

We will learn more about Gherkin in the subsequent lectures.

Next: Decomposition and boundaries



One ubiquitous language for the whole shop turns out to be impossible.

“Product” means one thing to marketing and another to the warehouse.

These boundaries can be identified with **bounded context**.

Summary

- **Strategic design** partitions the domain into subdomains, ranks them, and draws the boundaries.
- A **subdomain** is a fine-grained area of business activity.
- Every subdomain is **core**, **supporting**, or **generic**.
- Different types of subdomains decide the solution strategy: in-house, buy, or outsource.
- Subdomains are derived from the **requirements** and can be mapped to the **code structure**.
- The **ubiquitous language** is one vocabulary for experts, documents, and code.